Unit 4 AS2: Live Sessions Attendance and Participation

Vikram Nadathur

June 4, 2026

1 A Comprehensive Look at EVERY Python Data Viz Library

In this notebook we look at all the major plotting libraries in python.

[ ]:
```
!pip install blackcellmagic > /dev/null

# Load the new extension
%load_ext blackcellmagic
```

```
The blackcellmagic extension is already loaded. To reload it, use:
  %reload_ext blackcellmagic
```

[ ]:
```
# This cell previously attempted to install and load jupyterlab-black/lab_black.
# Due to persistent issues, we are now relying on `blackcellmagic` for␣
 ↪formatting.
# `blackcellmagic` was installed and loaded in cell `p8d2MA5_m4yg`.
```

[ ]:
```
def my_function(arg1, arg2):
    print("This will auto-format when run!")
```

1.0.1 Installing and Using a Code Formatter

To correctly install and use a code formatter in Google Colab, we will use `jupyterlab-black`. This package provides the `lab_black` IPython extension, which integrates the Black code formatter into your Jupyter/Colab environment.

[ ]:
```
# Install jupyterlab-black
# This package provides the 'lab_black' IPython extension for Black code␣
 ↪formatting.
!pip install jupyterlab-black > /dev/null

print('jupyterlab-black installed successfully.')
```

```
jupyterlab-black installed successfully.
```

After installation, you need to load the `lab_black` extension. Once loaded, any Python code cell you execute will be automatically formatted according to Black's style guidelines.

[ ]:
```
# This cell previously attempted to install and load jupyterlab-black/lab_black.
# Due to persistent issues, we are now relying on `blackcellmagic` for␣
 ↪formatting.
# `blackcellmagic` was installed and loaded in cell `p8d2MA5_m4yg`.

# To verify formatting, you can run any Python code cell, and `blackcellmagic`␣
 ↪should format it.
```

1.0.2 Example of Automatic Formatting

Run the cell below, and you'll see how `lab_black` automatically formats the code when executed.

[ ]:
```
# This code will be automatically formatted by lab_black when you run the cell.
def some_unformatted_function(arg1, arg2=None, arg3='default'):
  if arg2 is None:
    print('No arg2')
  else:
    print(f'Arg2 is {arg2}')
  # Fix: Ensure types are compatible for addition. Convert arg1 to string if␣
 ↪arg3 is string.
  # Or, make sure arg3 is a number if arg1 is a number and addition is intended.
  # For this example, let's assume arg3 was meant to be concatenated as a␣
 ↪string.
  return str(arg1) + str(arg3)


result = some_unformatted_function(10, 'hello')  # Example call
print(f"Result: {result}")

# Another example
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
if len(my_list) > 5 and my_list[0] == 1: print("List is long and starts with 1")
```

```
Arg2 is hello
Result: 10default
List is long and starts with 1
```

[ ]:
```
# Gotta import these.
import pandas as pd
import numpy as np
```

2 Matplotlib

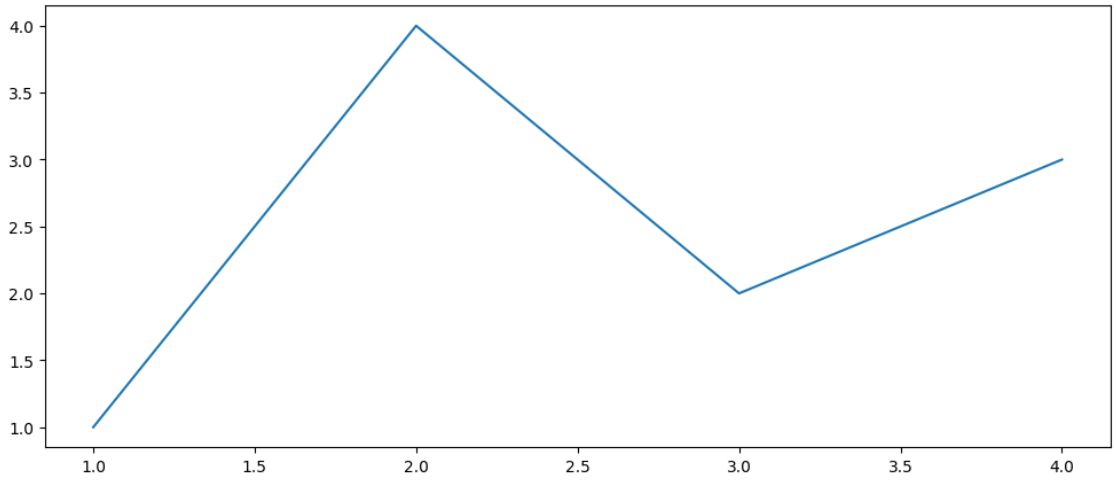
2.1 gitub stars: 14k

Positives: - Most solid plotting library in python. - Customization to do almost anything. - Widely used and other packages build off of it.

Negatives: - Hard learn at first. - Complex plots can be code heavy. - Not best for interactive plots.

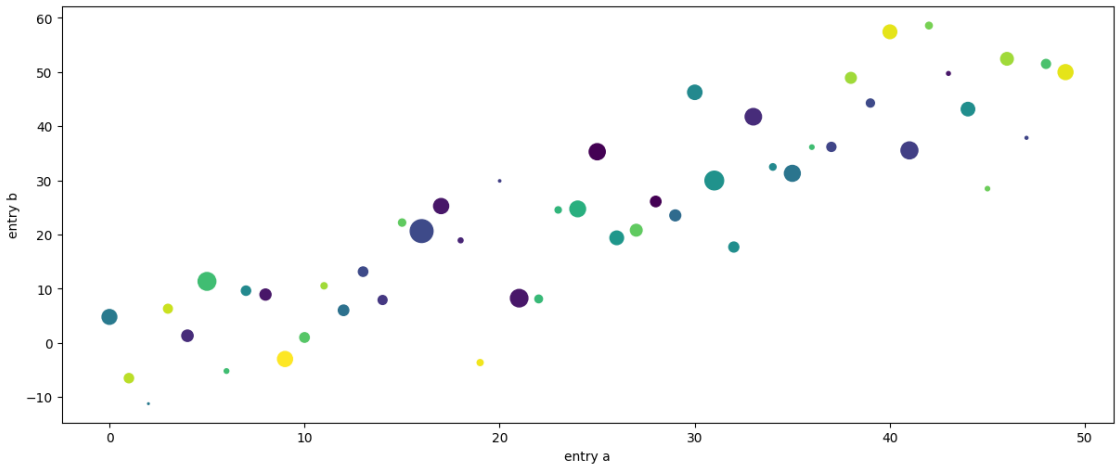
[ ]:
```
# The imports
import matplotlib as mpl
import matplotlib.pyplot as plt
```

[ ]:
```
fig, ax = plt.subplots(figsize=(12, 5))  # Create a figure containing a single␣
 ↪axes.
# Plot basic sequential coordinate data
ax.plot([1, 2, 3, 4], [1, 4, 2, 3])
# Plot some data on the axes.
plt.show()
```



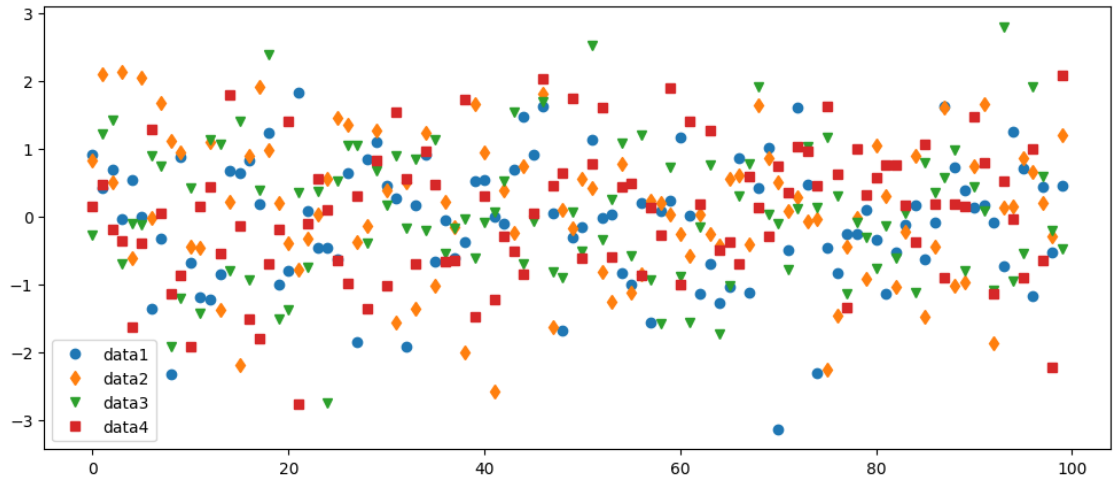
[ ]:
```
b = np.matrix([[1, 2], [3, 4]])
b_asarray = np.asarray(b)
np.random.seed(19680801)  # seed the random number generator.
data = {"a": np.arange(50), "c": np.random.randint(0, 50, 50), "d": np.random.
 ↪randn(50)}
data["b"] = data["a"] + 10 * np.random.randn(50)
data["d"] = np.abs(data["d"]) * 100

fig, ax = plt.subplots(figsize=(12, 5), layout="constrained")
ax.scatter("a", "b", c="c", s="d", data=data)
ax.set_xlabel("entry a")
ax.set_ylabel("entry b")
```

[ ]:
```
Text(0, 0.5, 'entry b')
```



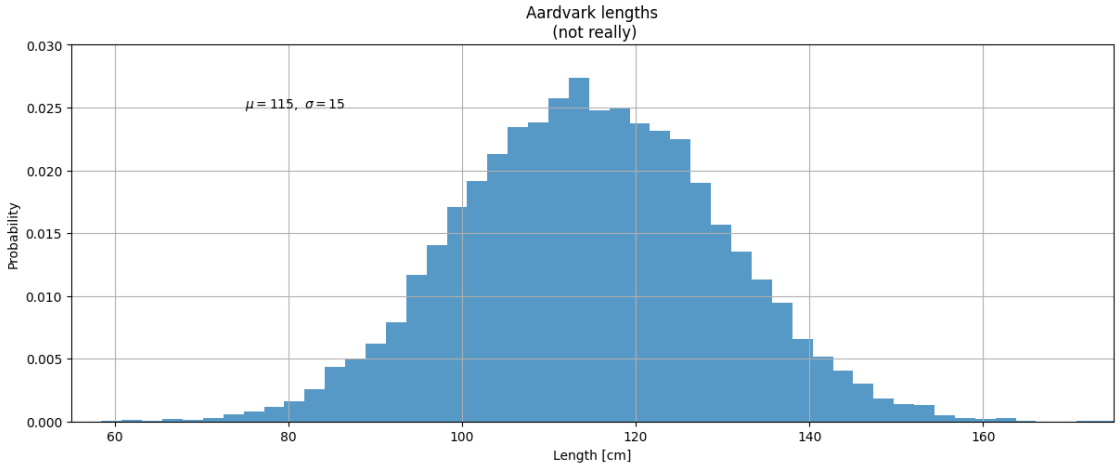
[ ]:
```
data1, data2, data3, data4 = np.random.randn(4, 100)  # make 4 random data sets
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(data1, "o", label="data1")
ax.plot(data2, "d", label="data2")
ax.plot(data3, "v", label="data3")
ax.plot(data4, "s", label="data4")
ax.legend()
```

[ ]:
```
<matplotlib.legend.Legend at 0x7d30d068cc20>
```



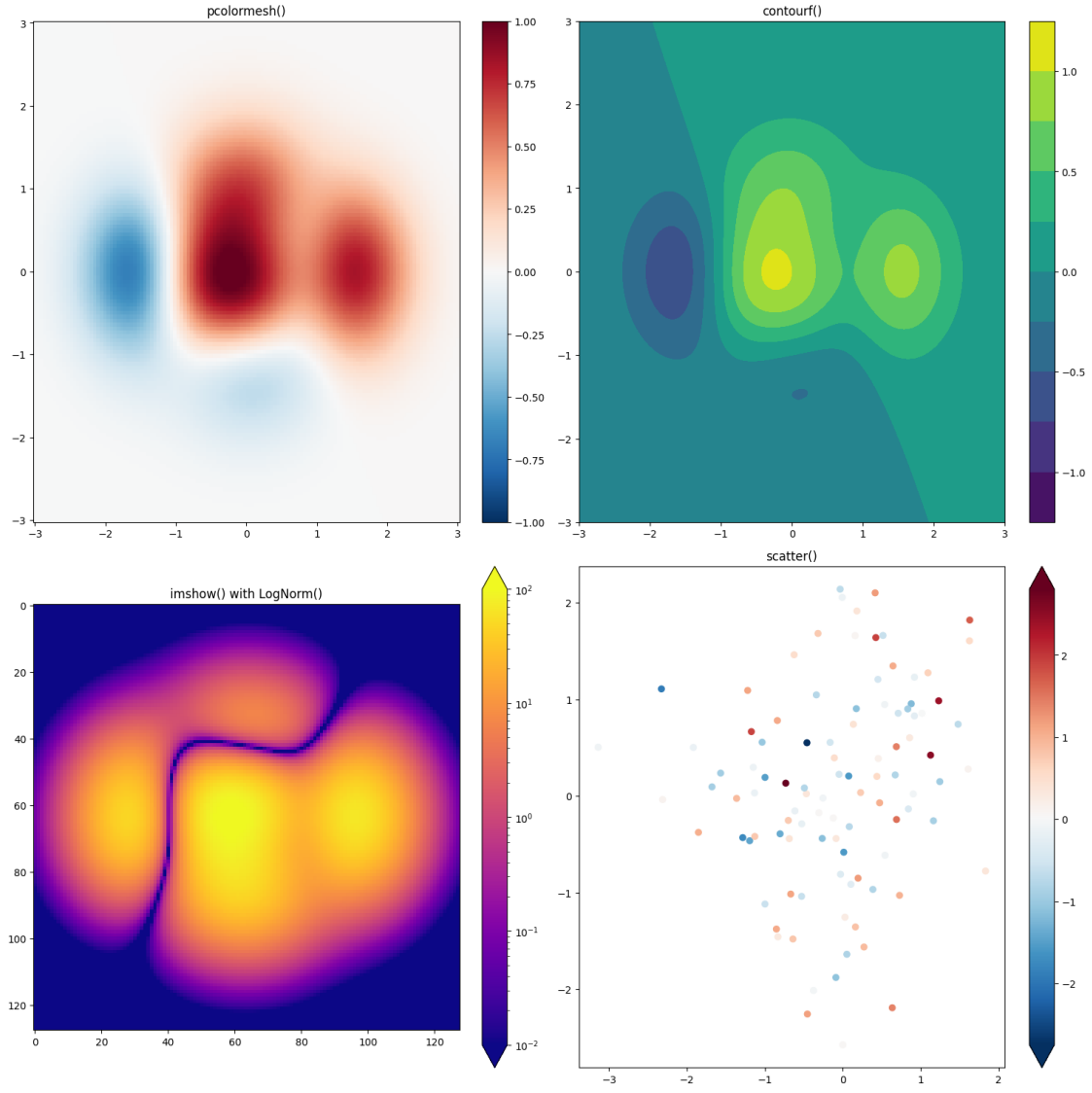
[ ]:
```
mu, sigma = 115, 15
x = mu + sigma * np.random.randn(10000)
fig, ax = plt.subplots(figsize=(12, 5), layout="constrained")
# the histogram of the data
```

```
n, bins, patches = ax.hist(x, 50, density=1, facecolor="C0", alpha=0.75)

ax.set_xlabel("Length [cm]")
ax.set_ylabel("Probability")
ax.set_title("Aardvark lengths\n (not really)")
ax.text(75, 0.025, r"$\mu=115,\ \sigma=15$")
ax.axis([55, 175, 0, 0.03])
ax.grid(True)
```



[ ]:
```
X, Y = np.meshgrid(np.linspace(-3, 3, 128), np.linspace(-3, 3, 128))
Z = (1 - X / 2 + X ** 5 + Y ** 3) * np.exp(-(X ** 2) - Y ** 2)

fig, axs = plt.subplots(2, 2, figsize=(15, 15), layout="constrained")
pc = axs[0, 0].pcolormesh(X, Y, Z, vmin=-1, vmax=1, cmap="RdBu_r")
fig.colorbar(pc, ax=axs[0, 0])
axs[0, 0].set_title("pcolormesh()")

co = axs[0, 1].contourf(X, Y, Z, levels=np.linspace(-1.25, 1.25, 11))
fig.colorbar(co, ax=axs[0, 1])
axs[0, 1].set_title("contourf()")

pc = axs[1, 0].imshow(
    Z ** 2 * 100, cmap="plasma", norm=mpl.colors.LogNorm(vmin=0.01, vmax=100)
)
fig.colorbar(pc, ax=axs[1, 0], extend="both")
axs[1, 0].set_title("imshow() with LogNorm()")

pc = axs[1, 1].scatter(data1, data2, c=data3, cmap="RdBu_r")
fig.colorbar(pc, ax=axs[1, 1], extend="both")
axs[1, 1].set_title("scatter()")
```

[ ]: Text(0.5, 1.0, 'scatter()')



3 Seaborn

3.1 github stars: 9k

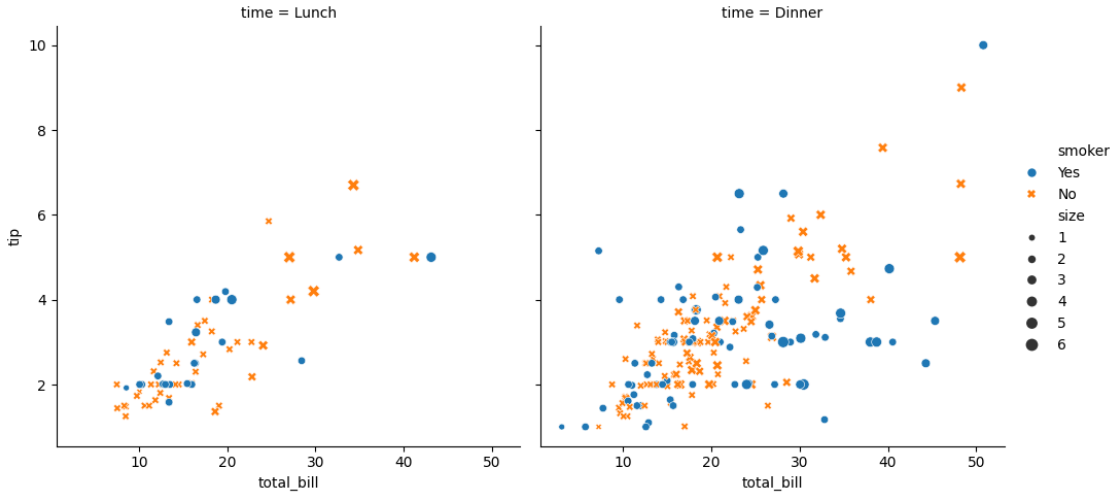
Positives: - Standard plots out of the box - Perfect for statisical anaylsis. - Fast to use for standard plots

Negatives: - Built on top of matplotlib - Less ability to customize

[ ]:
```
# The import
import seaborn as sns
```

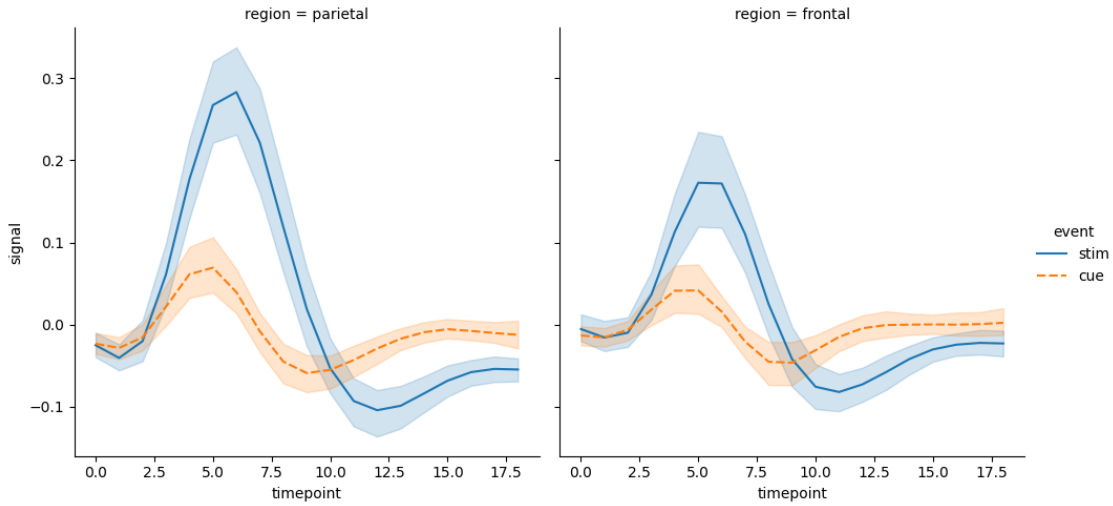
3.2 Relative Between two features

[ ]:
```
# Load an example dataset
tips = sns.load_dataset("tips")

# Create a visualization
sns.relplot(
    data=tips,
    x="total_bill",
    y="tip",
    col="time",
    hue="smoker",
    style="smoker",
    size="size",
)
plt.show()
```



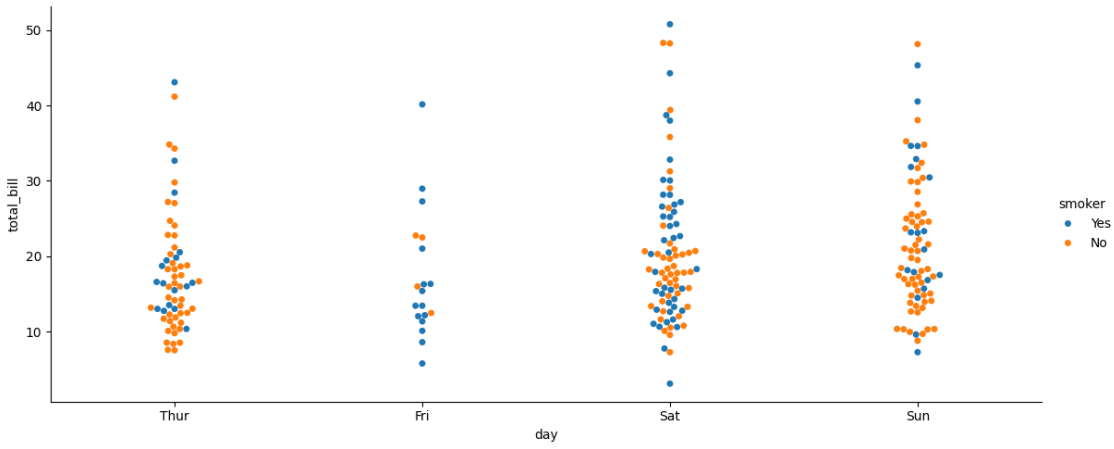
3.3 Statistical Estimation

[ ]:
```
fmri = sns.load_dataset("fmri")
sns.relplot(
    data=fmri,
    kind="line",
    x="timepoint",
    y="signal",
    col="region",
    hue="event",
    style="event",
)
plt.show()
```



3.4 Categorical Plot

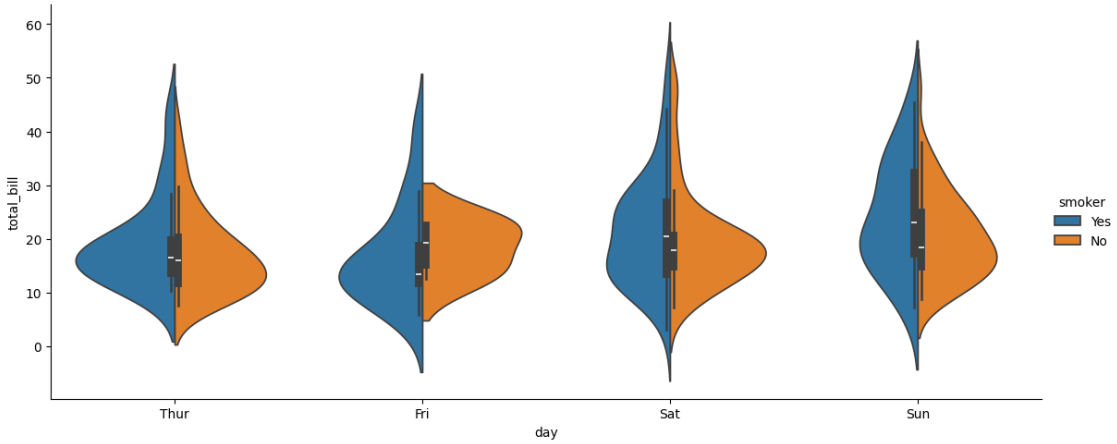
[ ]:
```
sns.catplot(
    data=tips, kind="swarm", x="day", y="total_bill", hue="smoker", height=5,␣
 ↪aspect=2.3
)
plt.show()
```



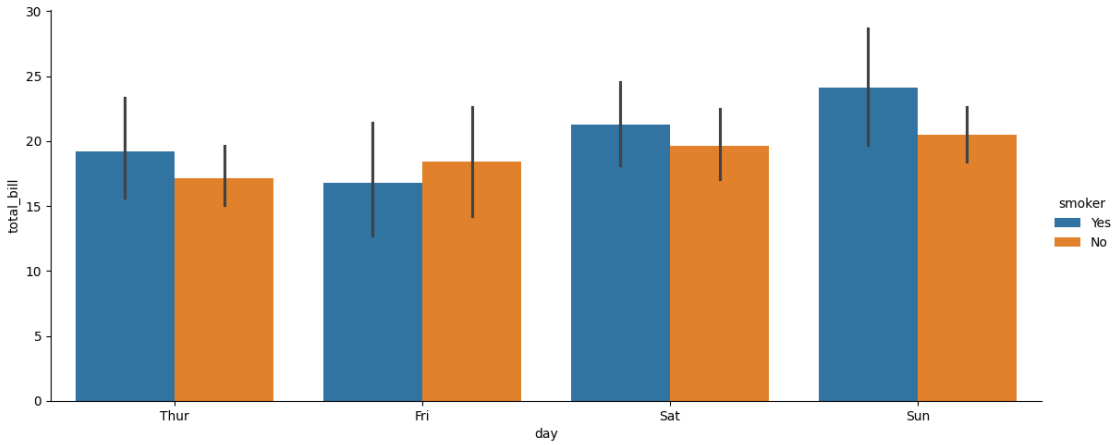
[ ]:
```
sns.catplot(
    data=tips,
    kind="violin",
    x="day",
    y="total_bill",
```

```
    hue="smoker",
    split=True,
    height=5,
    aspect=2.3,
)
```

[ ]: <seaborn.axisgrid.FacetGrid at 0x7d3096e10530>

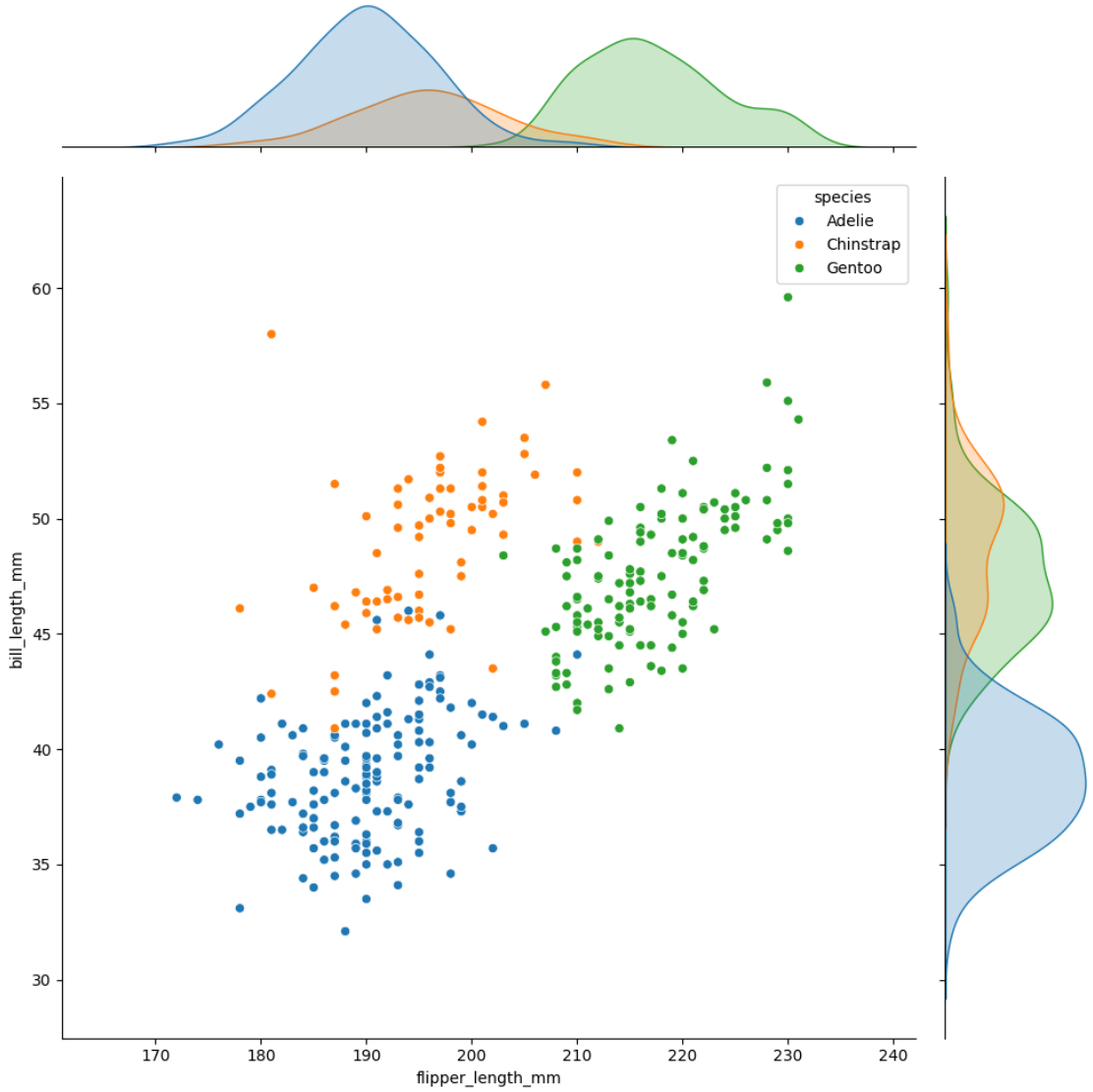


[ ]:
```
sns.catplot(
    data=tips, kind="bar", x="day", y="total_bill", hue="smoker", height=5,␣
 ↪aspect=2.3
)
plt.show()
```



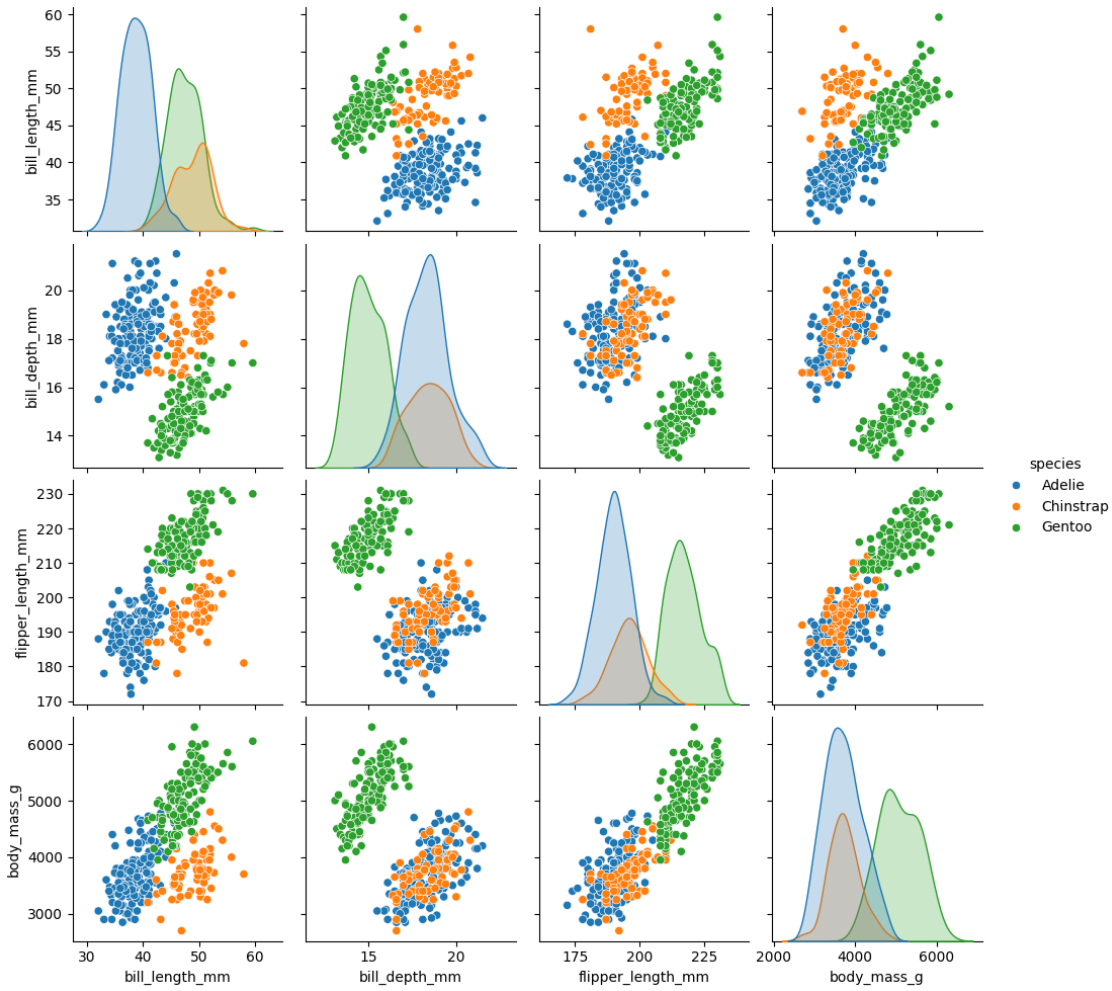
[ ]:
```
penguins = sns.load_dataset("penguins")
sns.jointplot(
    data=penguins, x="flipper_length_mm", y="bill_length_mm", hue="species",␣
 ↪height=10
)
```

[ ]: <seaborn.axisgrid.JointGrid at 0x7d3096b1af00>



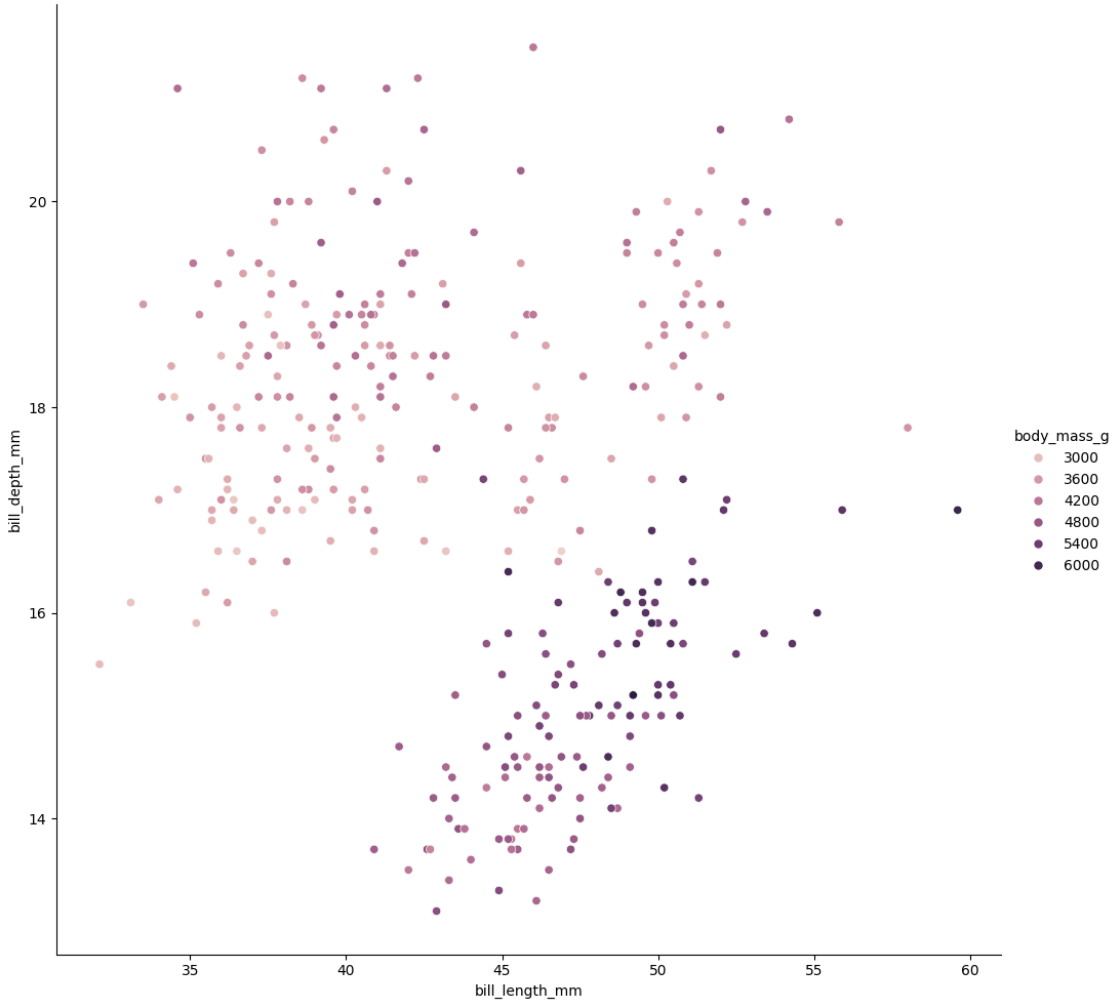
[ ]:
```
sns.pairplot(data=penguins, hue="species")
```

[ ]: <seaborn.axisgrid.PairGrid at 0x7d30986221e0>



[ ]:
```
sns.relplot(
    data=penguins, x="bill_length_mm", y="bill_depth_mm", hue="body_mass_g",␣
 ↪height=10
)
```

[ ]: <seaborn.axisgrid.FacetGrid at 0x7d30963ad790>



4 Bokeh

Positives: - Interactive plots made easy. - Lots of customization options. - Like matplotlib but interactive

Negatives: - like matplotlib but interactive - Code heavy

[ ]:
```
# import
from bokeh.plotting import figure
from bokeh.io import output_notebook, push_notebook, show

output_notebook()
```

[ ]:
```
# prepare some data
x = [1, 2, 3, 4, 5]
y = [6, 7, 2, 4, 5]
```

```
# create a new plot with a title and axis labels
p = figure(title="Simple line example", x_axis_label="x", y_axis_label="y")

# add a line renderer with legend and line thickness to the plot
p.line(x, y, legend_label="Temp.", line_width=2)

# show the results
show(p)
```

[ ]:
```
# prepare some data
x = [1, 2, 3, 4, 5]
y1 = [6, 7, 2, 4, 5]
y2 = [2, 3, 4, 5, 6]
y3 = [4, 5, 5, 7, 2]

# create a new plot with a title and axis labels
p = figure(title="Multiple line example", x_axis_label="x", y_axis_label="y")

# add multiple renderers
p.line(x, y1, legend_label="Temp.", color="blue", line_width=2)
p.line(x, y2, legend_label="Rate", color="red", line_width=2)
p.line(x, y3, legend_label="Objects", color="green", line_width=2)

# show the results
show(p)
```

[ ]:
```
from bokeh.layouts import row

# prepare some data
x = list(range(11))
y0 = x
y1 = [10 - i for i in x]
y2 = [abs(i - 5) for i in x]

# create three plots with one renderer each
s1 = figure(width=250, height=250, background_fill_color="#fafafa")
s1.circle(x, y0, size=12, color="#53777a", alpha=0.8)

s2 = figure(width=250, height=250, background_fill_color="#fafafa")
s2.triangle(x, y1, size=12, color="#c02942", alpha=0.8)

s3 = figure(width=250, height=250, background_fill_color="#fafafa")
s3.square(x, y2, size=12, color="#d95b43", alpha=0.8)

# put the results in a row that automatically adjusts
# to the browser window's width
```

```
show(row(children=[s1, s2, s3], sizing_mode="scale_width"))
```

```
BokehDeprecationWarning: 'circle() method with size value' was deprecated in
Bokeh 3.4.0 and will be removed, use 'scatter(size=…) instead' instead.
BokehDeprecationWarning: 'triangle() method' was deprecated in Bokeh 3.4.0 and
will be removed, use "scatter(marker='triangle', …) instead" instead.
BokehDeprecationWarning: 'square() method' was deprecated in Bokeh 3.4.0 and
will be removed, use "scatter(marker='square', …) instead" instead.
```

[ ]:
```
import numpy as np

from bokeh.plotting import figure, show

# generate some data
N = 1000
x = np.random.random(size=N) * 100
y = np.random.random(size=N) * 100

# generate radii and colors based on data
radii = y / 100 * 2
colors = ["#%02x%02x%02x" % (255, int(round(value * 255 / 100)), 255) for value␣
 ↪in y]

# create a new plot with a specific size
p = figure(
    title="Vectorized colors and radii example",
    sizing_mode="stretch_width",
    max_width=1200,
    height=400,
)

# add circle renderer
p.circle(
    x,
    y,
    radius=radii,
    fill_color=colors,
    fill_alpha=0.6,
    line_color="lightgrey",
)

# show the results
show(p)
```

[ ]:
```
from bokeh.layouts import layout
from bokeh.models import Div, RangeSlider, Spinner
from bokeh.plotting import figure, show
```

```
# 1. Prepare sample coordinate data points
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = [4, 5, 5, 7, 2, 6, 4, 9, 1, 3]

# 2. Initialize the plot figure with an initial x-axis range and dimensions
p = figure(x_range=(1, 9), width=500, height=250)
# Add large interactive circular markers to the plot
points = p.circle(x=x, y=y, size=30, fill_color="#21a7df")

# 3. Create a descriptive text header component using HTML/Div formatting
div = Div(
    text="""
          <p>Select the circle's size using this control element:</p>
          """,
    width=200,
    height=30,
)

# 4. Create a numeric adjustment spinner to alter the glyph marker size
spinner = Spinner(
    title="Circle size",
    low=0,
    high=60,
    step=5,
    value=points.glyph.size,
    width=200,
)
# Use client-side JavaScript linking to instantly update circle size when the␣
 ↪spinner changes
spinner.js_link("value", points.glyph, "size")

# 5. Create a range slider to manipulate the visible window of the x-axis
range_slider = RangeSlider(
    title="Adjust x-axis range",
    start=0,
    end=10,
    step=1,
    value=(p.x_range.start, p.x_range.end),
)
# Link the low (0) and high (1) bounds of the slider to the plot's x_range␣
 ↪start and end properties
range_slider.js_link("value", p.x_range, "start", attr_selector=0)
range_slider.js_link("value", p.x_range, "end", attr_selector=1)

# 6. Organize all structural elements and widgets into a clean grid/row layout␣
 ↪structure
```

```
layout = layout(
    [
        [div, spinner],
        [range_slider],
        [p],
    ]
)

# Render the interactive dashboard inside the notebook
show(layout)
```

```
BokehDeprecationWarning: 'circle() method with size value' was deprecated in
Bokeh 3.4.0 and will be removed, use 'scatter(size=…) instead' instead.
```

[ ]:
```
from bokeh.io import output_file, show
from bokeh.models import ColumnDataSource
from bokeh.palettes import Spectral6
from bokeh.plotting import figure

output_file("colormapped_bars.html")

fruits = ["Apples", "Pears", "Nectarines", "Plums", "Grapes", "Strawberries"]
counts = [5, 3, 4, 2, 4, 6]

source = ColumnDataSource(data=dict(fruits=fruits, counts=counts,␣
 ↪color=Spectral6))

p = figure(
    x_range=fruits,
    y_range=(0, 9),
    height=500,
    width=1000,
    title="Fruit counts",
    toolbar_location=None,
    tools="",
)

p.vbar(
    x="fruits",
    top="counts",
    width=0.9,
    color="color",
    legend_field="fruits",
    source=source,
)

p.xgrid.grid_line_color = None
```

```
p.legend.orientation = "horizontal"
p.legend.location = "top_center"

show(p)
```

[ ]:
```
pip install bokeh_sampledata
```

```
Requirement already satisfied: bokeh_sampledata in
/usr/local/lib/python3.12/dist-packages (2025.0)
Requirement already satisfied: icalendar in /usr/local/lib/python3.12/dist-
packages (from bokeh_sampledata) (7.1.2)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-
packages (from bokeh_sampledata) (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-
packages (from pandas>=1.2->bokeh_sampledata) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas>=1.2->bokeh_sampledata)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
packages (from pandas>=1.2->bokeh_sampledata) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
packages (from pandas>=1.2->bokeh_sampledata) (2026.2)
Requirement already satisfied: typing-extensions~=4.10 in
/usr/local/lib/python3.12/dist-packages (from icalendar->bokeh_sampledata)
(4.15.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.8.2->pandas>=1.2->bokeh_sampledata) (1.17.0)
```

[ ]:
```
import pandas as pd

from bokeh.io import output_file, show
from bokeh.models import (
    BasicTicker,
    ColorBar,
    ColumnDataSource,
    LinearColorMapper,
    PrintfTickFormatter,
)
from bokeh.plotting import figure
from bokeh.sampledata.unemployment1948 import data
from bokeh.transform import transform

output_file("unemploymemt.html")

data.Year = data.Year.astype(str)
data = data.set_index("Year")
data.drop("Annual", axis=1, inplace=True)
data.columns.name = "Month"
```

```
# reshape to 1D array or rates with a month and year for each row.
df = pd.DataFrame(data.stack(), columns=["rate"]).reset_index()

source = ColumnDataSource(df)

# this is the colormap from the original NYTimes plot
colors = [
    "#75968f",
    "#a5bab7",
    "#c9d9d3",
    "#e2e2e2",
    "#dfccce",
    "#ddb7b1",
    "#cc7878",
    "#933b41",
    "#550b1d",
]
mapper = LinearColorMapper(palette=colors, low=df.rate.min(), high=df.rate.
  ↪max())

p = figure(
    width=1000,
    height=500,
    title="US unemployment 1948-2016",
    x_range=list(data.index),
    y_range=list(reversed(data.columns)),
    toolbar_location=None,
    tools="",
    x_axis_location="above",
)

p.rect(
    x="Year",
    y="Month",
    width=1,
    height=1,
    source=source,
    line_color=None,
    fill_color=transform("rate", mapper),
)

color_bar = ColorBar(
    color_mapper=mapper,
    ticker=BasicTicker(desired_num_ticks=len(colors)),
    formatter=PrintfTickFormatter(format="%d%%"),
)
```

```
p.add_layout(color_bar, "right")

p.axis.axis_line_color = None
p.axis.major_tick_line_color = None
p.axis.major_label_text_font_size = "7px"
p.axis.major_label_standoff = 0
p.xaxis.major_label_orientation = 1.0

show(p)
```

[ ]:
```
import networkx as nx

from bokeh.io import output_file, show
from bokeh.models import (
    BoxSelectTool,
    Circle,
    EdgesAndLinkedNodes,
    HoverTool,
    MultiLine,
    NodesAndLinkedEdges,
    Plot,
    Range1d,
    TapTool,
)
from bokeh.palettes import Spectral4
from bokeh.plotting import from_networkx

G = nx.karate_club_graph()

plot = Plot(
    width=400, height=400, x_range=Range1d(-1.1, 1.1), y_range=Range1d(-1.1, 1.
 ↪1)
)
plot.title.text = "Graph Interaction Demonstration"

plot.add_tools(HoverTool(tooltips=None), TapTool(), BoxSelectTool())

graph_renderer = from_networkx(G, nx.circular_layout, scale=1, center=(0, 0))

graph_renderer.node_renderer.glyph = Circle(radius=0.15,␣
 ↪fill_color=Spectral4[0]) # Changed size to radius
graph_renderer.node_renderer.selection_glyph = Circle(radius=0.15,␣
 ↪fill_color=Spectral4[2]) # Changed size to radius
graph_renderer.node_renderer.hover_glyph = Circle(radius=0.15,␣
 ↪fill_color=Spectral4[1]) # Changed size to radius
```

```
graph_renderer.edge_renderer.glyph = MultiLine(
    line_color="#CCCCCC", line_alpha=0.8, line_width=5
)
graph_renderer.edge_renderer.selection_glyph = MultiLine(
    line_color=Spectral4[2], line_width=5
)
graph_renderer.edge_renderer.hover_glyph = MultiLine(
    line_color=Spectral4[1], line_width=5
)

graph_renderer.selection_policy = NodesAndLinkedEdges()
graph_renderer.inspection_policy = EdgesAndLinkedNodes()

plot.renderers.append(graph_renderer)

output_file("interactive_graphs.html")
show(plot)
```

5 Plotly Express

5.1 github stars: 657

Positives: - Quickly make interactive plots - Like seaborn but interactive - Maps are

Negatives: - Hard to customize - Some default plots (bar) look gross

[ ]:
```
# The import
import plotly.express as px
```

[ ]:
```
df = px.data.iris()
fig = px.scatter(
    df,
    x="sepal_width",
    y="sepal_length",
    color="species",
    size="petal_length",
    hover_data=["petal_width"],
)
fig.show()
```

[ ]:
```
df = px.data.tips()
fig = px.scatter(
    df, x="total_bill", y="tip", color="smoker", facet_col="sex",␣
 ↪facet_row="time"
)
fig.show()
```

[ ]:
```
df = px.data.gapminder().query("continent == 'Oceania'")
fig = px.line(df, x="year", y="lifeExp", color="country", markers=True)
fig.show()
```

[ ]:
```
df = px.data.tips()
fig = px.bar(df, x="sex", y="total_bill", color="smoker", barmode="group")
fig.show()
```

[ ]:
```
df = px.data.iris()
fig = px.scatter_matrix(
    df,
    dimensions=["sepal_width", "sepal_length", "petal_width", "petal_length"],
    color="species",
)
fig.show()
```

[ ]:
```
df = px.data.iris()
fig = px.parallel_coordinates(
    df,
    color="species_id",
    labels={
        "species_id": "Species",
        "sepal_width": "Sepal Width",
        "sepal_length": "Sepal Length",
        "petal_width": "Petal Width",
        "petal_length": "Petal Length",
    },
    color_continuous_scale=px.colors.diverging.Tealrose,
    color_continuous_midpoint=2,
)
fig.show()
```

[ ]:
```
df = px.data.tips()
fig = px.parallel_categories(
    df, color="size", color_continuous_scale=px.colors.sequential.Inferno
)
fig.show()
```

[ ]:
```
df = px.data.gapminder()
fig = px.scatter(
    df.query("year==2007"),
    x="gdpPercap",
    y="lifeExp",
    size="pop",
    color="continent",
    hover_name="country",
    log_x=True,
```

```
    size_max=60,
)
fig.show()
```

[ ]:
```
df = px.data.gapminder().query("year == 2007")
fig = px.sunburst(
    df,
    path=["continent", "country"],
    values="pop",
    color="lifeExp",
    hover_data=["iso_alpha"],
)
fig.show()
```

[ ]:
```
df = px.data.tips()
fig = px.violin(
    df, y="tip", x="smoker", color="sex", box=True, points="all", hover_data=df.
 ↪columns
)
fig.show()
```

[ ]:
```
df = px.data.carshare()
fig = px.scatter_mapbox(
    df,
    lat="centroid_lat",
    lon="centroid_lon",
    color="peak_hour",
    size="car_hours",
    color_continuous_scale=px.colors.cyclical.IceFire,
    size_max=15,
    zoom=10,
    mapbox_style="carto-positron",
)
fig.show()
```

[ ]:
```
df = px.data.election()
fig = px.scatter_3d(
    df,
    x="Joly",
    y="Coderre",
    z="Bergeron",
    color="winner",
    size="total",
    hover_name="district",
    symbol="result",
    color_discrete_map={"Joly": "blue", "Bergeron": "green", "Coderre": "red"},
)
```

```
fig.show()
```

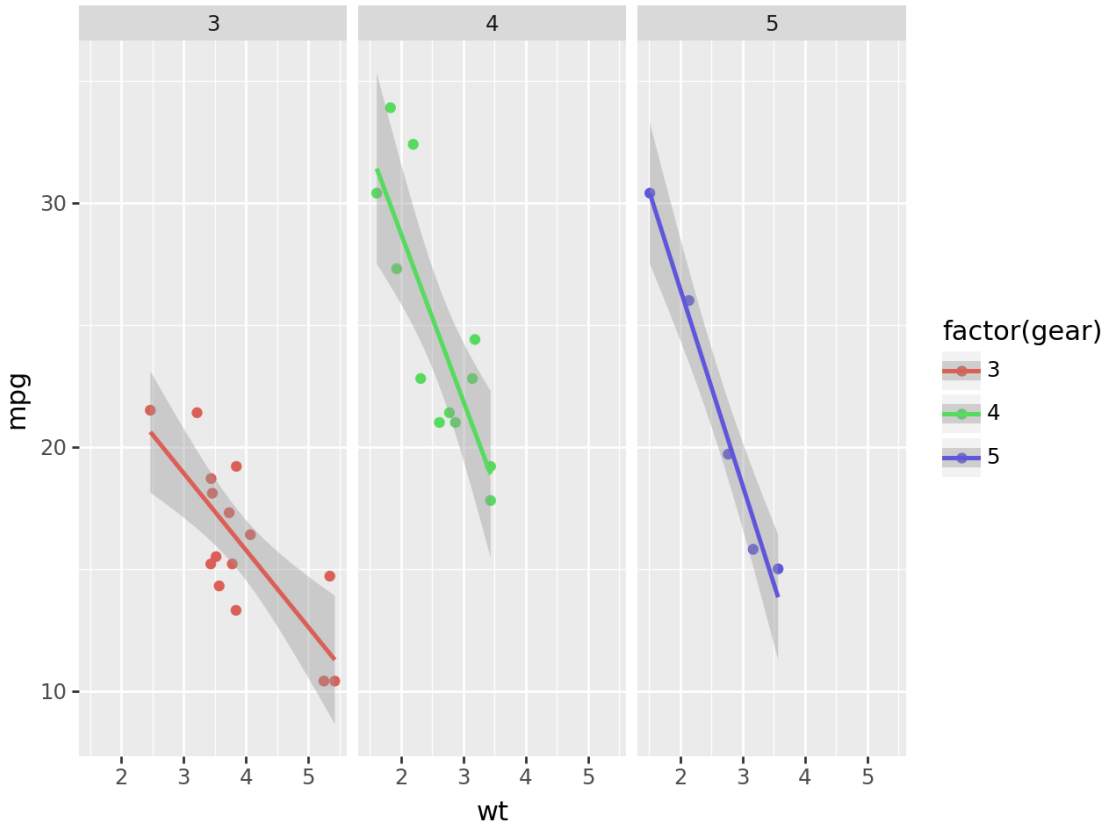
6 plotnine (ggplot)

6.1 github stars: 2.9k

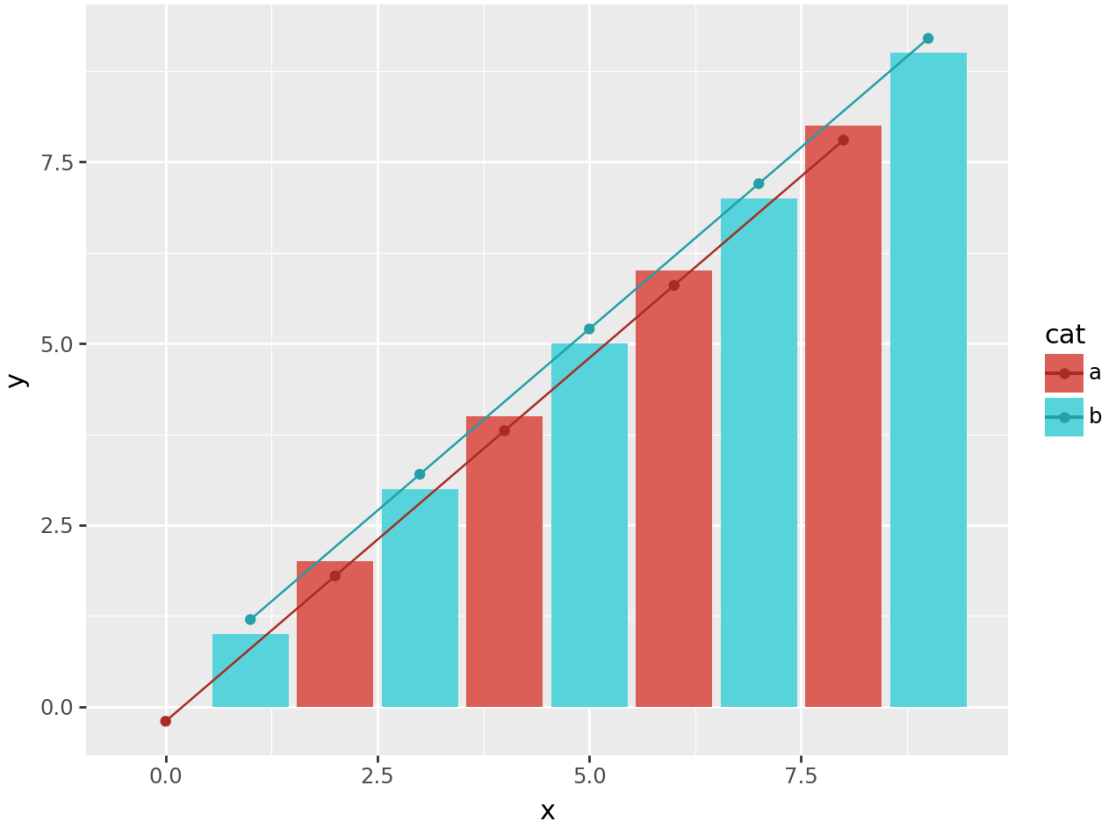
Positives: - Pretty! - Based on ggplot for R lovers.

Negatives: - Unique coding syntax - Not as developed (bad docs)

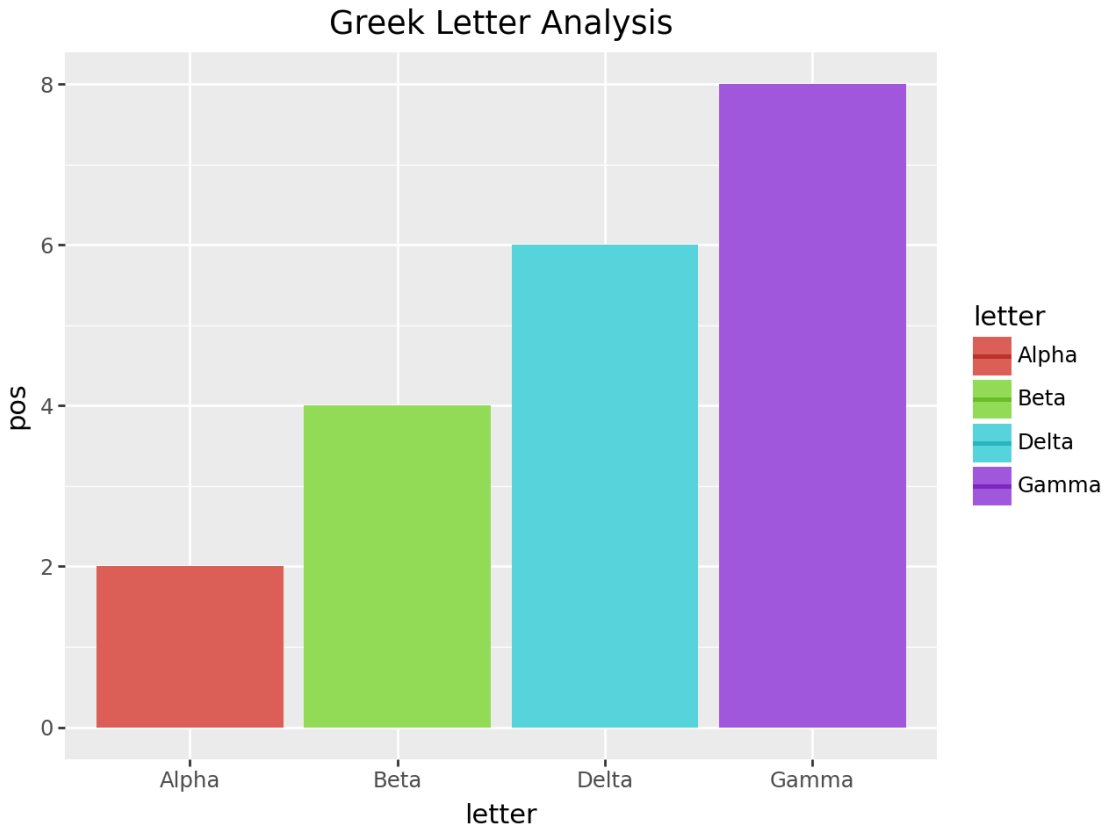
[ ]:
```
from plotnine import ggplot, geom_point, aes, stat_smooth, facet_wrap
from plotnine.data import mtcars

(
    ggplot(mtcars, aes("wt", "mpg", color="factor(gear)"))
    + geom_point()
    + stat_smooth(method="lm")
    + facet_wrap("~gear")
)
```



[ ]:
```
from plotnine import geom_col, geom_path, scale_color_discrete

n = 10
df = pd.DataFrame(
    {
        "x": np.arange(n),
        "y": np.arange(n),
        "yfit": np.arange(n) + np.tile([-0.2, 0.2], n // 2),
        "cat": ["a", "b"] * (n // 2),
    }
)

(
    ggplot(df)
    + geom_col(aes("x", "y", fill="cat"))
    + geom_point(aes("x", y="yfit", color="cat"))
    + geom_path(aes("x", y="yfit", color="cat"))
    + scale_color_discrete(l=0.4)  # new
)
```



[ ]:
```
from plotnine import geom_line, scale_color_hue, ggtitle

df2 = pd.DataFrame(
    {
        "letter": ["Alpha", "Beta", "Delta", "Gamma"] * 2,
        "pos": [1, 2, 3, 4] * 2,
        "num_of_letters": [5, 4, 5, 5] * 2,
    }
)

(
    ggplot(df2)
    + geom_col(aes(x="letter", y="pos", fill="letter"))
    + geom_line(aes(x="letter", y="num_of_letters", color="letter"), size=1)
    + scale_color_hue(l=0.45)  # some contrast to make the lines stick out
    + ggtitle("Greek Letter Analysis")
)
```



7 Altair

7.1 github stars: 7.1k

Positives: - Declarative Visualization

Negatives: - Declarative Visualization

From their docs: **We believe that these challenges can be addressed without the creation of yet another visualization library that has a programmatic API and built-in rendering. Altair's approach to building visualizations uses a layered design that leverages the full capabilities of existing visualization libraries:**

[ ]:
```
!pip install vega_datasets > /dev/null
```

[ ]:
```
import altair as alt

# load a simple dataset as a pandas DataFrame
from vega_datasets import data

cars = data.cars()

alt.Chart(cars).mark_point().encode(
    x="Horsepower",
    y="Miles_per_Gallon",
    color="Origin",
)
```

[ ]:
```
alt.Chart(…)
```

8 Pandas

Pandas is a plotting library? Yes! It uses matplotlib backed.

Positives: - Plot directly from a dataframe. No other imports. - Suprisingly has a lot of plotting options. - Great starting point for matplotlib plots

Negatives: - Not interactive - Plain and simple

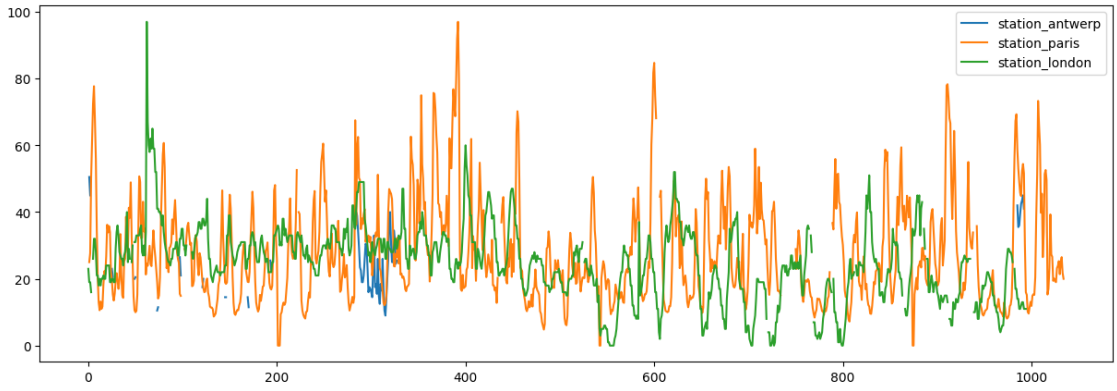
[ ]:
```
!wget https://raw.githubusercontent.com/pandas-dev/pandas/master/doc/data/
↪air_quality_no2.csv > /dev/null
```

```
--2026-06-04 14:20:48--  https://raw.githubusercontent.com/pandas-
dev/pandas/master/doc/data/air_quality_no2.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)…
185.199.108.133, 185.199.109.133, 185.199.110.133, …
Connecting to raw.githubusercontent.com
(raw.githubusercontent.com)|185.199.108.133|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 31984 (31K) [text/plain]
Saving to: 'air_quality_no2.csv'
```

```
air_quality_no2.csv 100%[===================>]  31.23K  --.-KB/s    in 0.01s

2026-06-04 14:20:48 (3.13 MB/s) - 'air_quality_no2.csv' saved [31984/31984]
```

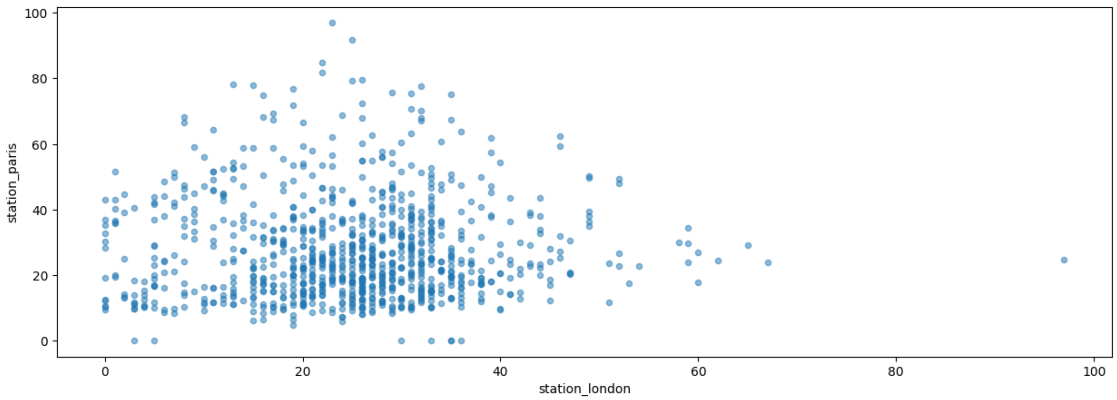
[ ]:
```
import pandas as pd

air_quality = pd.read_csv('air_quality_no2.csv', engine='python')
air_quality.plot(figsize=(15, 5))
```

[ ]:
```
<Axes: >
```



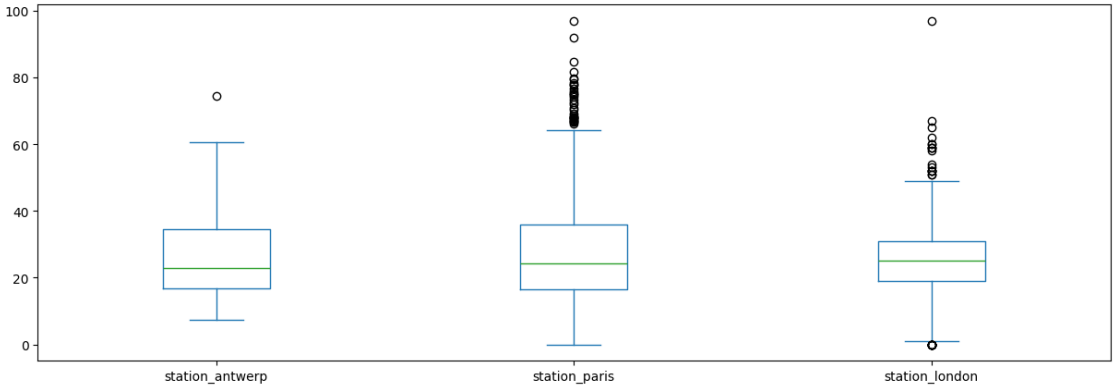
[ ]:
```
air_quality.plot.scatter(x="station_london", y="station_paris", alpha=0.5,␣
↪figsize=(15, 5))
```

[ ]:
```
<Axes: xlabel='station_london', ylabel='station_paris'>
```



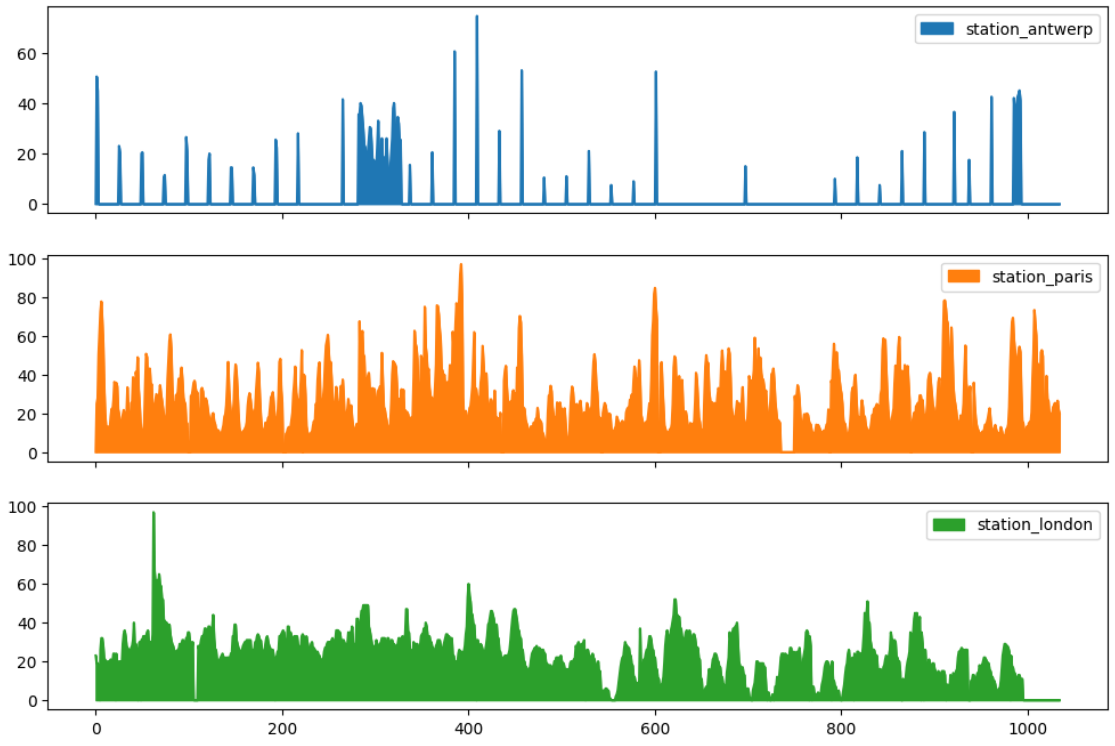
[ ]:
```
air_quality.plot.box(figsize=(15, 5))
```

[ ]: <Axes: >



[ ]:
```
air_quality.plot.area(figsize=(12, 8), subplots=True)
```

[ ]: array([<Axes: >, <Axes: >, <Axes: >], dtype=object)



9 What did I miss?

Let me know in the comments!